Since 1983

Dr. Vishwanath Karad
MIT WORLD PEACE UNIVERSITY | PUNE
TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

MIT-WPU
॥ विश्वशान्तिर्भुवं ध्रुवा ॥

MPFT: Multipath Reliable File Transfer over Parallel TCP Paths

Mini-project presentation

Computer Networks Laboratory (CSE20110)
Department of Computer Engineering and Technology

Saksham Singh Sisodiya – PRN 1262242908
Parth Dande – PRN 1262242629
Anupria Gupta – PRN 1262241349

Group 1 Panel J Batch J1
Faculty: Dr. Uma Pujeri
Academic Year: 2026–2027

Problem and Objectives

PROBLEM

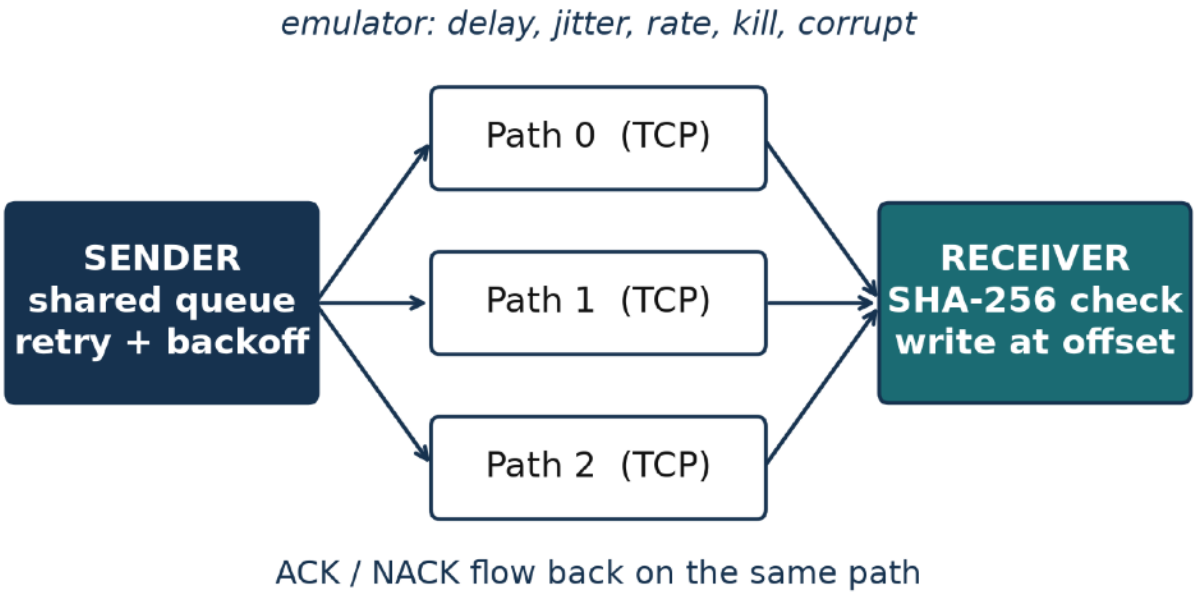
One TCP connection uses one network path and starts again from zero if it breaks, so extra links sit idle and a lost connection wastes the whole transfer [1][2].

Users: anyone moving large files between hosts with several links. **Scope:** one file, one sender, one receiver, emulated paths on one machine.

OBJECTIVES

1. Send one file in chunks over **N parallel TCP paths** using a shared work queue.
2. **Detect corruption** (header CRC, chunk and file SHA-256) and repair it by retransmission.
3. **Survive a dead or silent path** and resume interrupted transfers.

Proposed Design and Tools



TOOLS

Python 3.15, standard library
macOS, pytest
FastAPI dashboard
Wireshark Lua dissector

PROTOCOLS

TCP per path
Own 20-byte header + CRC-32
SHA-256 per chunk and file [5]

ADDRESSING

Receiver 0.0.0.0:9000
Emulated paths 127.0.0.1:9201–9203
Dashboard 127.0.0.1:8080

Implementation and Working

SCHEDULER (WORK STEALING)

```
queue = PriorityQueue()
path_worker(p):  # 1 thread/path
  i = queue.get()
  send CHUNK i, wait <= 5 s
  ACK  -> record bytes, RTT
  NACK/timeout/reset ->
         requeue(i)
requeue(i): max 5 tries,
  backoff, retry goes first
```

WORKING

1 Sender hashes file once

2 Paths pull chunks from queue

3 Receiver checks SHA-256, writes at offset

4 Whole-file hash, rename, DONE

IMPLEMENTATION

protocol.py, sender.py, receiver.py, emulator.py, status.py (about 970 lines)

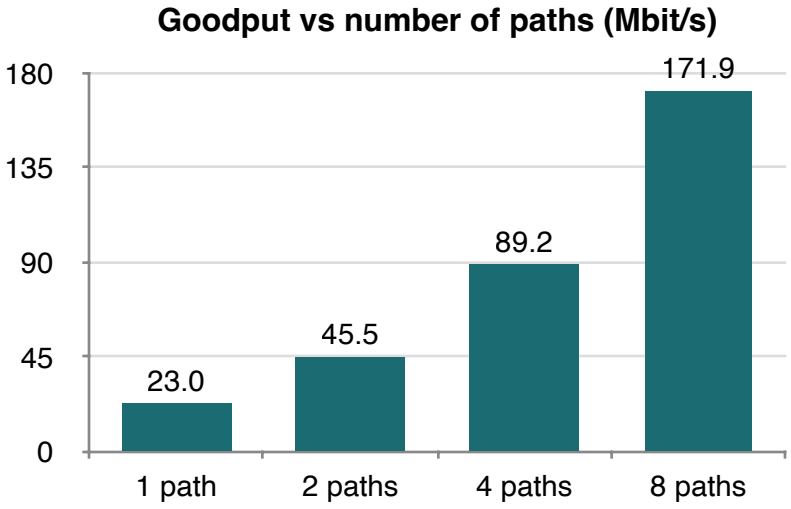
Dashboard in web/

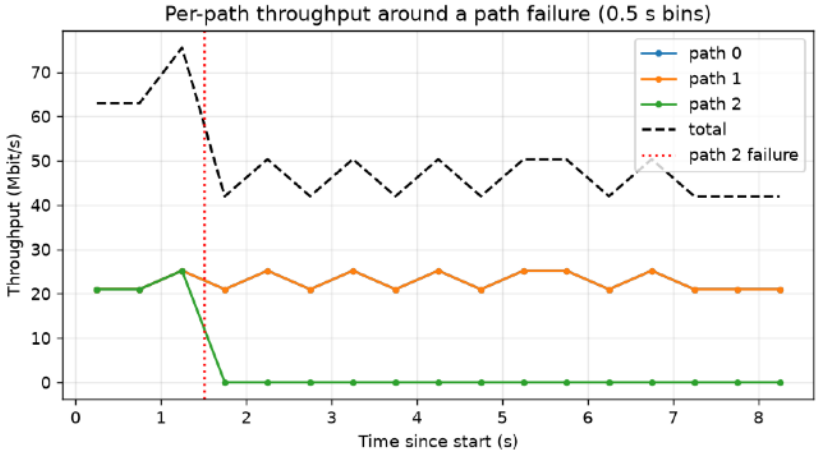
TROUBLESHOOTING

Recovery took 6.9 s: retries waited behind fresh chunks.

Fix: priority queue, retries first. **Result:** 0.15 s.

Testing and Results





TEST

50 MB file, 40 Mbit/s paths: 1–8 paths; path 2 killed at 1.5 s; 0/1/5 % corrupted chunks

EXPECTED

Goodput grows with paths; hash matches after failure or corruption

OBSERVED

23.0 to 171.9 Mbit/s; failure run succeeded, recovery 0.15 s; 5 % corruption: 12 of 200 chunks resent

OBJECTIVES ACHIEVED

O1–O5. 79 automated tests pass (53 unit, 26 browser)

Working Demonstration

FUNCTION TO DEMONSTRATE

Send a file over 3 emulated paths while one path dies; the receiver still gets a verified file.

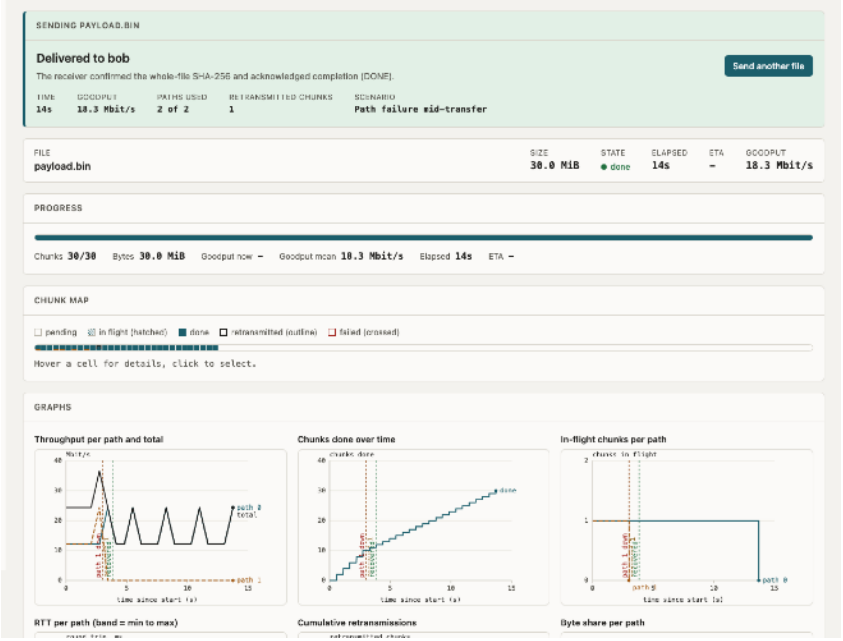
1. **Launch:** Start receiver, emulator (3 paths, one dies at 6 s) and dashboard
   `mpft receive | mpft emulate | mpft dashboard`

2. **Send:** Run the transfer over the three paths
   `mpft send big.bin --to 127.0.0.1:9201,...:9202,...:9203`

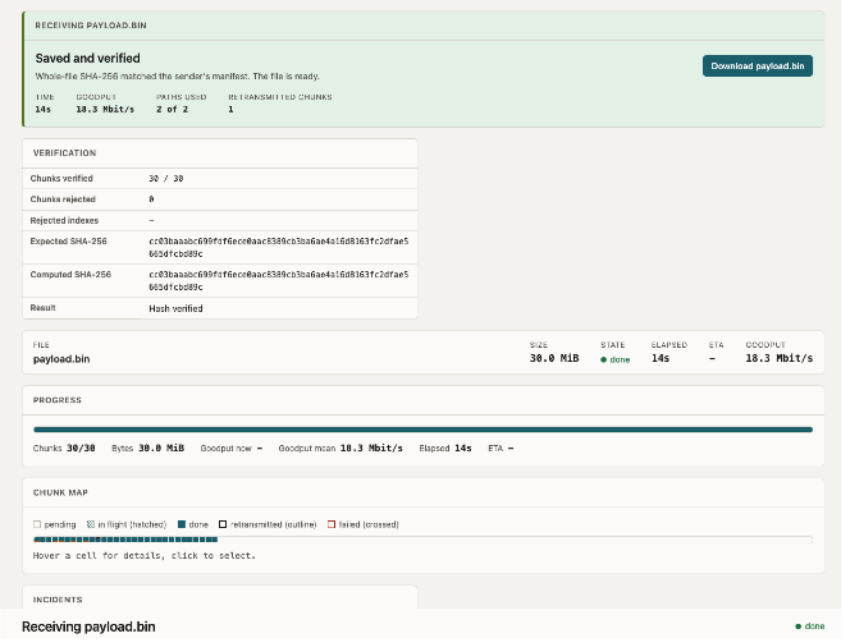
3. **Show:** Watch the dashboard: path 2 goes down, its chunk is resent on another path
   `hash status turns verified`

Success criterion: sender reports success, a path is marked dead, 1 or more retransmissions, and the output SHA-256 equals the input's.

Views and Scenarios



Sender: Delivered, send another



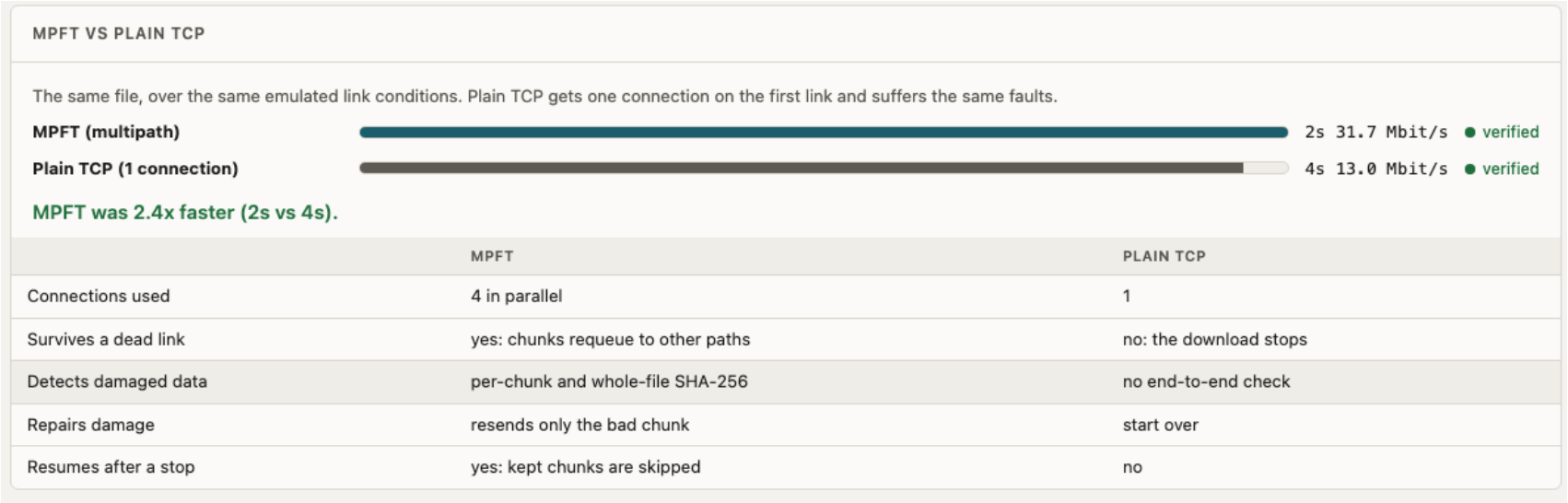
Receiver: verified, Download

Eight scenarios: clean, slow, asymmetric, lossy, heavy loss, path failure, path flap, total failure (nothing saved, no download).

Graphs now use real time: throughput from chunk completions, millisecond axis under 2 s.

Receiver Download is on the page from the start (active once verified); Save as... picks the folder and file name.

MPFT versus Plain TCP



	MPFT	PLAIN TCP
Connections used	4 in parallel	1
Survives a dead link	yes: chunks requeue to other paths	no: the download stops
Detects damaged data	per-chunk and whole-file SHA-256	no end-to-end check
Repairs damage	resends only the bad chunk	start over
Resumes after a stop	yes: kept chunks are skipped	no

WHAT IS COMPARED

Same file, same emulated link.

Plain TCP: one connection, one hash at the end, no resume.

RESULT OF ONE RUN

6 MiB, four 16 Mbit/s paths:

MPFT 2 s, plain TCP 4 s (2.4x faster); both files verified.

WHEN PLAIN TCP WINS

Clean unthrottled link: one connection can be faster.

Lossy or dead link: plain TCP fails, MPFT repairs.

Conclusion and Contributions

OUTCOME

Working multipath transfer with chunk and file verification, failure recovery and resume

APPLICATION

Large files over several links (Wi-Fi plus Ethernet, LAN plus VPN)

LIMITATION

One chunk in flight per path; tested on emulated paths on one machine only

FUTURE SCOPE

Sliding window per path; real two-machine test with Wireshark captures

TEAM CONTRIBUTIONS

Saksham Singh Sisodiya
Protocol, sender, receiver, resume, web UI

Parth Dande
Emulator, experiments, presentation

Anupria Gupta
Wireshark, dashboard views, test report

References

[1] J. Eddy, “Transmission Control Protocol (TCP),” RFC 9293, IETF, 2022.

[2] M. Allman, V. Paxson and E. Blanton, “TCP Congestion Control,” RFC 5681, IETF, 2009.

[3] A. Ford et al., “TCP Extensions for Multipath Operation with Multiple Addresses,” RFC 8684, IETF, 2020.

[4] C. Raiciu et al., “How hard can it be? Designing and implementing a deployable Multipath TCP,” USENIX NSDI, 2012.

[5] NIST, “Secure Hash Standard (SHS),” FIPS PUB 180-4, 2015.

Cited on slides 2–3: TCP [1][2], multipath background [3][4], SHA-256 [5].